

Kanban Board Vishal Tutte

In this document I will go through the functions that I have made in this project, and my understanding of them

1. Taking input

I created two functions, `menu()` and `takeInputMenu()` for this part. `Menu` is a very simple function, it simply prints the options. I decided to split this from the `takeInputMenu` as it looks cleaner to me.

`takeInputMenu()` is a simple function as well, it just has many parts. It first takes in the number 1-6 for which it has some safety, to ensure it is a number 1-6.

Then it has a `switch` statement which is the main part of the input.

```
switch (input){
    case 1:
        DisplayBoard(head_ptr);
        break;
    case 2:
        printf("Enter file name: ");
        while ((getchar()) != '\n'); // to clear any \ns that may break the fgets
        fgets(filename, sizeof(filename), stdin);
        filename[strcspn(filename, "\n")] = 0; // Remove trailing newline
        loadBoardFromFile(&head_ptr, filename);
        break;
    case 3:
        edit_list(head_ptr);
        break;
    case 4:
        printf("Options:\n");
        printf("1. Edit the name of a list\n2. Add a new list\n3. Delete a list\n4. F
        printf("Enter your option: ");
        scanf("%d", &input2);
        while (input2 < 1 || input2 > 4){
            printf("Invalid Input, it must be an integer (1 - 4)\n");
            printf("Enter your option (1 - 4): ");
            scanf("%d", &input2);
```

```

    }
    if(input2 == 4){
        break;
    }
    edit_board(&head_ptr, input2);
    break;
case 5:
    printf("Enter filename: ");
    scanf("%24s", filename);
    saveBoardToFile(head_ptr, filename);
    break;
case 6:
    return 6;
    break;
default:
    printf("Error taking input\n");
    break;
}

```

1. Just displays the board. We have it so that in the main function we already start the program with the predefined values in the project specifications.
2. It takes the file name, but we use the `while ((getchar()) != '\n');` to remove the trailing characters from taking other inputs. This ensures there are no issues with taking in the filename, which could be especially dangerous considering we are dealing with memory.
3. Calls our `edit_list` function
4. We decided to take the type of input in the `switch` case itself as we don't have to deal with finding if the file name or list name is valid.
5. We take the `filename` in the `switch` and call the function
6. we return 6 from the function, which is the stopping case of the entire program.

2. Files

I was tasked with the two file functions, `loadBoardFromFile` and `saveBoardToFile` .

```

void loadBoardFromFile(List **head, const char *filename) {
    // opening file
    FILE *file = fopen(filename, "r");
    if (!file) {
        printf("Error opening file %s\n", filename);
        return;
    }

    if (*head != NULL) {
        freeAllLists(*head);
    }

    char line[100];
    List *currentList = NULL;

    // fgets reads a line at a time from the file into line
    while (fgets(line, sizeof(line), file)) {
        if (line[0] == '\n') continue; // Used to skip empty lines

        if (line[0] != '\t') { // If the line doesn't begin with a tab or an empty line then
            line[strcspn(line, ":\n")] = 0; // remove ':' and newline when storing the line
            addList(head, line); // calls the function to create a node with the new list
            // move currentList pointer to the newly added list
            currentList = *head;
            while (currentList->nextList != NULL) // moves to the next available position
                currentList = currentList->nextList;
        } else {
            // if it's an item
            char *itemName = line + 1; // pointer that skips tab
            itemName[strcspn(itemName, "\n")] = 0; // strip newline
            addItem(currentList, currentList->name, itemName); // adds the task to the list
        }
    }

    fclose(file);
    printf("\nBoard loaded successfully.\n");
    DisplayBoard(*head);
}

```

```
return;  
}
```

We call the helper function `freeAllLists` to ensure we're not just appending the list onto the already existing lists.

We then take each line as input from the file. We have it coded so that it can identify if it is a empty line, a list name, and an item name. This is done by checking the starting character. A `\n` means empty, and `\t` means item and neither means list name.

```
void saveBoardToFile(List *head, const char *filename) {  
    FILE *file = fopen(filename, "w");  
    if (!file) {  
        printf("Error opening file\n");  
        return;  
    }  
    // The items in the linked list continue to be printed until there is no more list  
    while (head != NULL) {  
        fprintf(file, "%s\n", head->name);  
        Item *item = head->item;  
        while (item != NULL) {  
            fprintf(file, "\t%s\n", item->name);  
            item = item->nextItem;  
        }  
        fprintf(file, "\n");  
        head = head->nextList;  
    }  
  
    fclose(file);  
    printf("Board has been saved.\n");  
    return;  
}
```

This function runs simple conditions. Keep printing until the next item in the list is empty, and print in the form

List:

item

item2

List2:

item3

item4

.

.

.

3. Freeing of memory

```
void freeAllLists(List *head) { //This is used to delete all the lists
    while (head != NULL) {
        List *tempList = head;
        head = head->nextList;

        // Free all items in the list
        Item *item = tempList->item;
        while (item != NULL) {
            Item *tempItem = item;
            item = item->nextItem;
            free(tempItem);
        }

        // Free the list node itself
        free(tempList);
    }
}
```

This function was coded by me mainly for the load board function, as well as for the end, when we are meant to prevent memory leaks when exiting the code, by freeing up all the list and item pointers we used `malloc` for.

4. Adding items/lists

```
void addList(List **head, const char *name) {
    List *newList = malloc(sizeof(List));
    strcpy(newList->name, name);
    newList->item = NULL; // initialising the values.
    newList->nextList = NULL;

    if (*head == NULL) { // If the list is empty, make the new list node the head
        *head = newList;
    } else { // Else add the new list at the end.
        List *current = *head;
        while (current->nextList != NULL){
            current = current->nextList;
        }
        current->nextList = newList;
    }
}
```

```
void addItem(List *head, const char *listName, const char *itemName) {
    while (head != NULL) {
        if (strcmp(head->name, listName) == 0) {
            Item *newItem = malloc(sizeof(Item));
            strcpy(newItem->name, itemName);
            newItem->nextItem = NULL;

            if (head->item == NULL) { // If it is the first item added to the list
                head->item = newItem;
            } else { // If it isn't, shuffle until the curr->nextItem is free
                Item *current = head->item;
                while (current->nextItem != NULL){
                    current = current->nextItem;
                }
                current->nextItem = newItem; // Then store it there.
            }
            return;
        } // if the head isn't the list name needed then go to the next name.
    }
```

```

        head = head->nextList;
    }
    printf("List name does not exist.\n");
    return;
}

```

Both of these functions are very similar.

They both:

- Use `malloc` to create new nodes (either `List` or `Item` nodes)
- Handle empty list cases as special conditions
- Traverse to the end of their respective lists to add new elements
- Initialise the new node's '`next`' pointer to `NULL`

However, there are key differences:

`addList` Function:

- Takes a double pointer (`**head`) to allow modification of the head pointer
- Simpler traversal as it only needs to find the end of one list
- Doesn't need to search for a specific list name

`addItem` Function:

- Takes a single pointer to head
- Has two levels of traversal: first finding the correct list, then finding the end of the items
- Requires list name matching using `strcmp`

The `addList` function follows a straightforward approach: if the board is empty, make the new list the head; otherwise, find the last list and append. The `addItem` function is more complex as it must first locate the correct list by name, then apply similar append logic for the items within that list.